Title Page

Swarm Behaviour Classification with Machine Learning

(A Supervised Learning Approach for High-Dimensional Swarm Behavior Data)

Daniel Sanchez

CSCI 460 – Introduction to Applied Machine Learning

Comprehensive Final Project

Submission Date: May 4, 2026

Abstract

This project applies supervised machine learning to the UCI Swarm Behaviour dataset to classify high-dimensional swarm movement patterns. The dataset contains three related binary classification tasks: Aligned, Flocking, and Grouped behavior. Each task includes approximately 24,000 observations and 2,400 numerical features describing agent positions, movement, and interaction patterns, with a binary Class target indicating whether the behavior is present. The main objective was to determine which machine learning model could provide the best balance of predictive generalization, and computational efficiency.

The project followed a complete machine learning workflow, including data cleaning, exploratory data analysis, preprocessing, dimensionality reduction analysis, model training, hyperparameter tuning, and final evaluation on held-out test data. Exploratory analysis showed that the datasets were high-dimensional but highly compressible, with PCA and effective-rank diagnostics suggesting a compact underlying structure. Six supervised models were implemented and tuned using stratified cross-validation: Logistic Regression, Ridge Classifier, Linear Support Vector Classifier, Decision Tree, Random Forest, and XGBoost. Model performance was evaluated using accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices, and runtime measurements. The final results showed near-perfect classification performance across the datasets, with the tuned Ridge Classifier without PCA selected as the best overall model because it combined excellent test performance with fast prediction time and low model complexity.

1. Introduction

1.1 Problem Definition

Swarm behavior refers to the collective movement and coordination patterns that emerge when many individual agents interact with one another. These agents may represent animals, robots, drones, or simulated particles that follow local movement rules. In this project, the prediction task is to classify whether a specific swarm behavior is present based on high-dimensional numerical measurements of agent movement and interaction.

This project focuses on three related binary classification tasks from the Swarm Behaviour dataset: Aligned, Flocking, and Grouped. Each task predicts whether a particular collective behavior is present or absent. The target variable is Class, where a value of 1 indicates that the behavior is present and a value of 0 indicates that the behavior is not present. Because each dataset contains only two possible target labels, the project is treated as a supervised binary classification problem.

The main machine learning challenge is to determine whether the movement, velocity, alignment, separation, cohesion, and neighborhood-count features of swarm agents contain enough predictive structure to accurately classify the behavior type. The project also evaluates which supervised machine learning model provides the best balance between predictive performance, generalization ability, and computational efficiency.

1.2 Dataset Description

The dataset selected for this project is the Swarm Behaviour dataset from the UCI Machine Learning Repository. The dataset is designed for modeling and predicting collective movement and coordination patterns in swarm systems. It contains three separate CSV files: Aligned, Flocking, and Grouped. Each file represents a separate binary classification task for one type of swarm behavior.

After cleaning, the Aligned dataset contained 24,016 instances and 2,401 columns. The Flocking dataset contained 24,015 instances and 2,401 columns after one malformed row was removed. The Grouped dataset contained 24,016 instances and 2,401 columns. In each dataset, 2,400 columns are numerical feature variables, and the final column is the binary target variable, Class.

The feature variables represent movement and interaction measurements for multiple swarm agents, also called boids. The main feature groups include x- and y-position values, x- and y-velocity values, alignment-vector values, separation-vector values, cohesion-vector values, and neighborhood-count values. These feature groups are repeated across 200 agents, which creates the high-dimensional structure of the dataset. For example, features such as x1, y1, xVel1, and yVel1 describe the position and velocity of agent 1, while similar feature groups are repeated for agents 2 through 200.

The target variable, Class, identifies whether the behavior associated with each dataset is present. In the Aligned dataset, Class indicates whether the agents are aligned. In the Flocking dataset, Class indicates whether the agents are flocking. In the Grouped dataset, Class indicates whether

the agents are grouped. Therefore, the three datasets share the same feature structure but represent three different binary classification objectives.

1.3 Objective and Research Questions

The main objective of this project is to build, tune, evaluate, and compare supervised machine learning models for classifying high-dimensional swarm behavior data. The project aims to identify the model that achieves the strongest overall performance while also remaining practical in terms of runtime and complexity.

This project is guided by the following research questions:

1. Can supervised machine learning models accurately classify aligned, flocking, and grouped swarm behavior using high-dimensional numerical movement features?
2. How do different model families compare on this dataset, including linear models, support vector machines, decision trees, Random Forest, and XGBoost?
3. Does dimensionality reduction with Principal Component Analysis preserve enough information to maintain strong classification performance?
4. Which tuned model provides the best balance of test-set performance, generalization, prediction time, and simplicity?

1.4 Report Structure

The remainder of this report is organized as follows. Section 2 presents the exploratory data analysis, including dataset shape, target-class balance, feature distributions, correlations, outlier patterns, and PCA-based structure. Section 3 describes the methodology used for data cleaning, preprocessing, feature analysis, model selection, experimental setup, hyperparameter tuning, and evaluation metrics. Section 4 presents the model results and discussion, including tuned model comparisons, final model selection, confusion matrices, ROC curves, runtime analysis, and interpretation of the results. Section 5 concludes the report by summarizing the key findings, answering the research questions, discussing limitations, and proposing future work.

2. Exploratory Data Analysis

2.1 Purpose of the Exploratory Analysis

Exploratory data analysis was performed before model training to understand the structure, quality, and modeling challenges of the Swarm Behaviour datasets. This step was important because machine learning performance depends heavily on whether the available features contain useful information for the prediction task. The course text emphasizes that a machine learning project should begin by understanding the task, the data representation, and whether the extracted features can support the intended prediction problem (Müller & Guido, 2016, Ch. 1). Therefore, the EDA phase focused on dataset size, data types, missing values, class balance, feature distributions, outliers, correlations, and dimensionality-reduction behavior.

The EDA also helped guide later modeling decisions. In particular, the analysis was used to determine whether stratified splitting was needed, whether scaling should be applied, whether dimensionality reduction with Principal Component Analysis (PCA) was worth testing, and whether evaluation should rely on metrics beyond accuracy.

2.2 Dataset Structure After Cleaning

After the initial cleaning process, all three datasets had the same target column name, Class, and all remaining values were numeric. The Flocking file originally contained a trailing space in the target column name and one malformed non-numeric value in the x1 column. After stripping column-name whitespace, coercing columns to numeric values, and removing the malformed row, the cleaned datasets were ready for EDA.

Table: Cleaned dataset dimensions

Dataset	Instances	Total Columns	Feature Columns	Target Column
Aligned	24,016	2,401	2,400	Class
Flocking	24,015	2,401	2,400	Class
Grouped	24,016	2,401	2,400	Class

Each dataset contained 2,400 numerical predictor features and one binary target column. The feature columns followed the same structure across the three datasets, which made the datasets directly comparable. This structure also matched the general machine learning representation described in the course text: each row is a sample or data point, and each column is a feature that describes the sample (Müller & Guido, 2016, Ch. 1). In this project, each row represents one swarm-state observation, and the feature columns represent movement and interaction measurements across multiple agents.

The data-type inspection showed that each cleaned dataset contained 2,000 floating-point columns and 401 integer columns. Because the datasets were already numerical after cleaning, no categorical encoding was required during EDA or preprocessing. This was an important finding because it allowed the project to focus on scaling, dimensionality reduction, and model comparison rather than one-hot encoding.

2.3 Summary Statistics and Feature Ranges

Summary statistics were computed for all three datasets to inspect feature ranges, central tendency, and variation. The results showed that the datasets had nearly identical feature structures and very similar numerical distributions. For example, position features such as x1 and y1 had wide ranges, while velocity features such as xVel1 and yVel1 had smaller but still meaningful numerical ranges. Alignment features generally stayed near the range of approximately -1 to 1, while separation and cohesion features contained many zero or near-zero values with occasional large values.

Table: Summary statistics for selected Aligned features.

Feature	Mean	Standard Deviation	Minimum	Median	Maximum	Interpretation
x1	108.31	842.18	-1414.14	156.09	1406.08	Agent x-position has a wide spatial range.
y1	-73.08	572.55	-1012.18	-156.69	1015.80	Agent y-position also varies widely.

xVel1	-0.74	6.45	-18.59	-0.46	18.47	Velocity is lower magnitude than position but still variable.
yVel1	0.04	7.37	-18.41	-1.60	18.63	Vertical velocity has a wide spread around zero.
xA1	-0.15	0.38	-1.00	0.00	1.00	Alignment vectors are bounded and centered near zero.
yA1	0.15	0.59	-1.00	0.00	1.04	Alignment values are bounded but variable.
xS1	-1.22	27.76	-944.07	0.00	68.56	Separation values are sparse with extreme movement cases.
yS1	-0.84	21.67	-847.91	0.00	234.74	Separation values show strong skew and occasional large values.

These summary statistics suggested that the dataset contained different feature scales and different distribution shapes. Since several algorithms studied in the course, especially support vector machines and some linear models, are affected by feature scaling, this finding supported the later use of scaling inside model pipelines. Chapter 3 of the course text explains that preprocessing and scaling are commonly used to adjust feature representations so that they are more suitable for machine learning algorithms (Müller & Guido, 2016, Ch. 3).

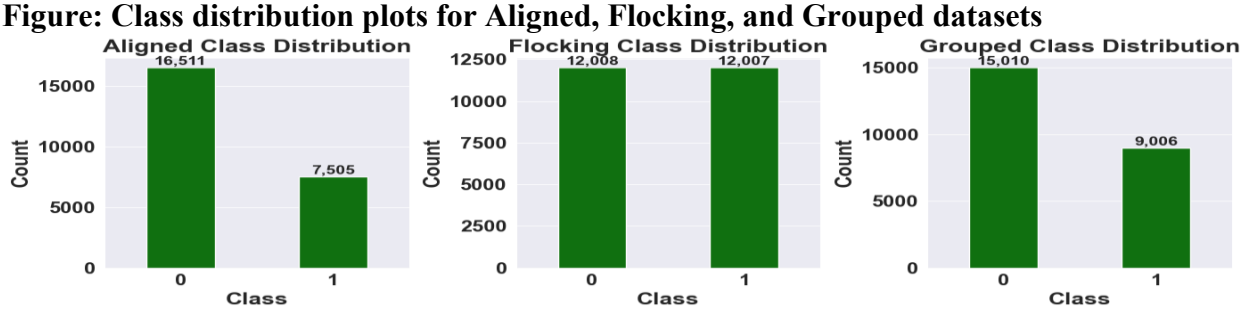
2.4 Target Class Balance

Class distribution plots were created for all three datasets to check whether the binary target variable was balanced. This step was important because class imbalance can make accuracy misleading if a model mainly predicts the majority class. The class-count results are shown in the table below.

Table: Class distribution by dataset. Approx*

Dataset	Class 0 Count	Class 0 Percent	Class 1 Count	Class 1 Percent	Balance Interpretation
Aligned	16,511	68.75%	7,505	31.25%	Imbalanced toward Class 0
Flocking	12,008	50.00%	12,007	50.00%	Nearly perfectly balanced
Grouped	15,010	62.50%	9,006	37.50%	Moderately imbalanced toward Class 0

Figure: Class distribution plots for Aligned, Flocking, and Grouped datasets



The Flocking dataset was almost perfectly balanced, with 12,008 observations in Class 0 and 12,007 observations in Class 1. In contrast, the Aligned and Grouped datasets were imbalanced toward Class 0. Aligned contained 68.75% Class 0 and 31.25% Class 1, while Grouped contained 62.50% Class 0 and 37.50% Class 1. This class imbalance was not severe enough to prevent classification, but it was important enough to affect the choice of evaluation metrics. As a result, later model evaluation used F1-score, precision, recall, and ROC-AUC in addition to accuracy. This decision is consistent with the course text's warning that accuracy alone can be limited for imbalanced classification problems and that additional classification metrics can provide a more complete performance picture (Müller & Guido, 2016, Ch. 5).

2.5 Feature Distribution Visualizations

Histograms were created for a selected set of shared feature columns across all three datasets. The first 15 feature columns matched across Aligned, Flocking, and Grouped, confirming that the datasets used the same feature layout. These histograms showed that the features did not all follow the same distribution. Some features had broad distributions, while several other features showed strong skewness and many repeated values near zero.

Figure 2.2. Histogram of selected flocking features.

The histogram patterns suggested that the swarm features captured different types of information. Position and velocity features represented continuous movement across space, while alignment, cohesion, separation, and neighborhood-count features reflected local interaction behavior. The presence of skewed distributions and repeated zero values also indicated that some feature groups were sparse or only activated under certain swarm conditions.

These findings anticipated two modeling challenges. First, models that depend on distances or coefficient magnitudes could be affected by the different scales of the features. Second, the high number of sparse or low-variation features suggested that dimensionality reduction or regularization might improve efficiency without removing class-separating information.

2.6 Outlier Analysis

An outlier analysis was performed using the interquartile range (IQR) rule for each of the 2,400 feature columns in all three datasets. The goal was not to automatically delete outliers, but to understand whether extreme values were widespread and whether they appeared to be part of the natural swarm dynamics. The results are summarized in the table below.

Table: IQR-based outlier summary.

Dataset	Total Feature Columns	Features with Zero Outliers	Features with At Least One Outlier	Percent of Features with Outliers
Aligned	2,400	726	1,674	69.75%
Flocking	2,400	724	1,676	69.83%
Grouped	2,400	726	1,674	69.75%

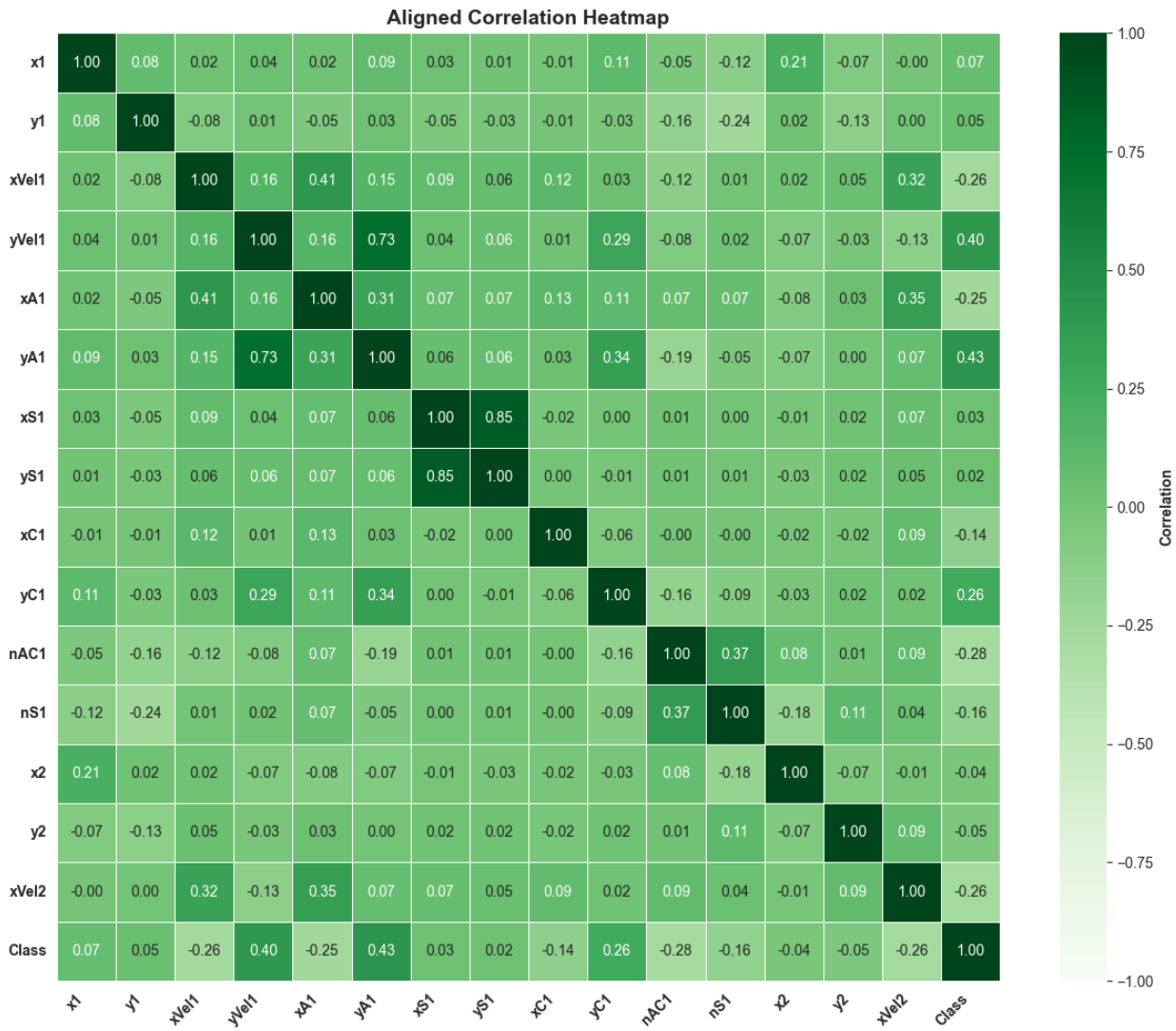
The outlier analysis showed that many features contained values outside the IQR bounds. For example, 1,674 of the 2,400 features in the Aligned and Grouped datasets contained at least one outlier, while 1,676 features in the Flocking dataset contained at least one outlier. However, these values were not treated as automatic data-entry errors. In swarm behavior data, extreme movement and interaction values may represent valid agent behavior, such as sudden separation, strong local cohesion, or unusual neighborhood interactions. Therefore, the EDA supported a conservative approach: clean clear malformed records, but avoid deleting large numbers of valid observations only because they were flagged by the IQR rule.

The outlier findings also influenced preprocessing. Chapter 3 discusses how scaling methods such as StandardScaler and RobustScaler can be used to bring features onto comparable scales, and it specifically notes that outliers can affect some scaling techniques (Müller & Guido, 2016, Ch. 3). In this project, the outlier analysis supported the later use of standardized preprocessing within model pipelines, while the final decision not to remove outliers preserved the original behavior patterns in the dataset.

2.7 Correlation Analysis

Correlation heatmaps were created using selected shared feature columns and the Class target. Because the full dataset had 2,400 features, plotting every feature in a correlation heatmap would be visually unreadable. Instead, a smaller representative set of shared features was used to inspect correlation patterns and identify potential redundancy.

Figure: Correlation heatmap for selected features and target variable.



The correlation heatmaps showed that some features were strongly correlated, while many others had weak correlations with one another and with the target variable. This pattern is expected in a high-dimensional swarm dataset because many features are repeated across agents and represent related movement concepts. For example, different position, velocity, and interaction-vector measurements can move together when the swarm exhibits coordinated behavior.

The presence of correlated features suggested that the dataset contained redundancy. This finding supported the later testing of PCA and regularized linear models. PCA is appropriate for this kind of investigation because it creates a transformed representation of the feature space and can summarize high-dimensional data using fewer components. Chapter 3 describes dimensionality reduction as a common unsupervised transformation for high-dimensional data, especially when the goal is to summarize important structure with fewer features (Müller & Guido, 2016, Ch. 3).

2.8 PCA-Based Structure and Dimensionality Findings

Because each dataset contained 2,400 feature columns, PCA was used as an advanced exploratory tool to inspect whether the high-dimensional feature space contained lower-

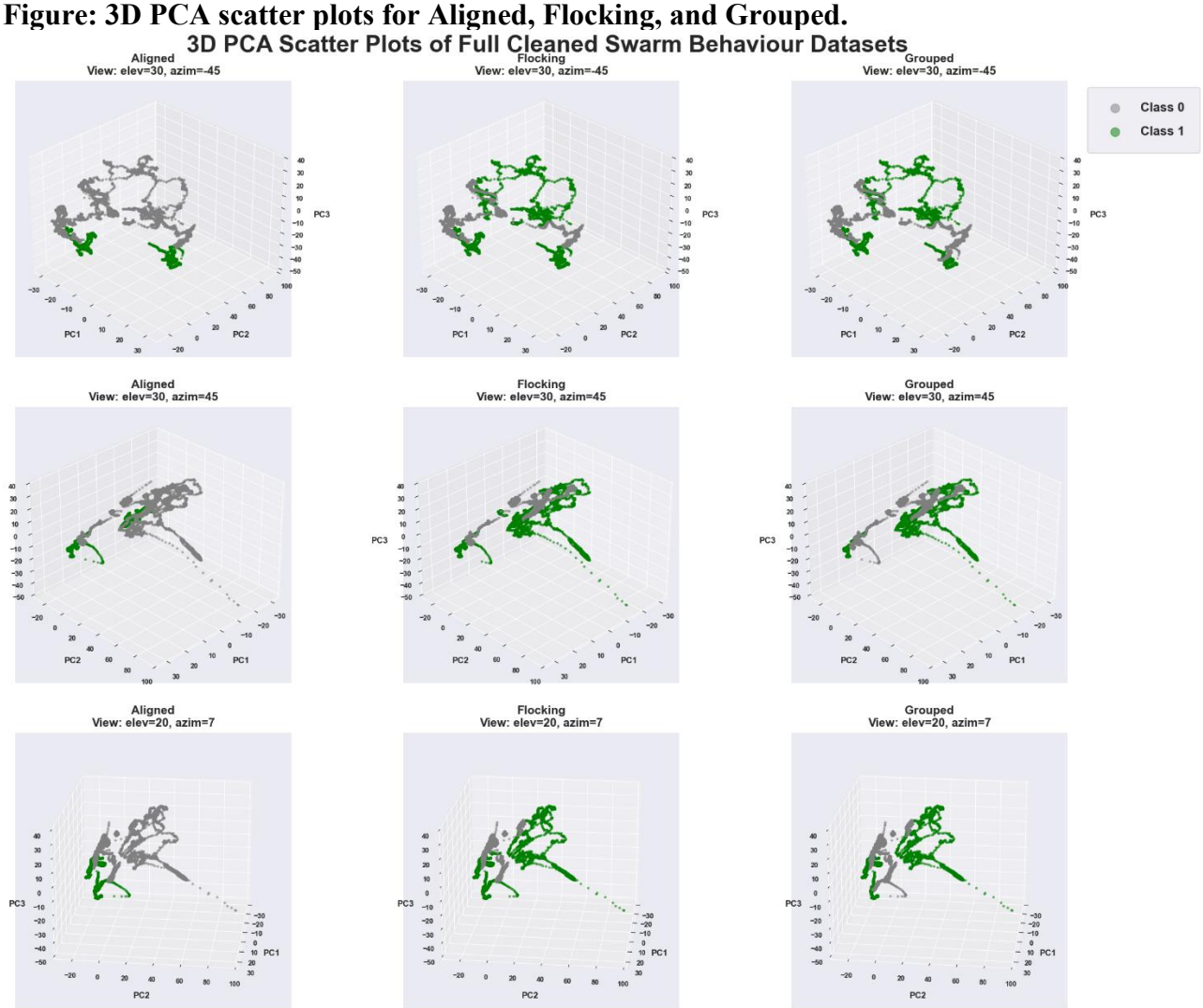
dimensional structure. PCA was not used at this stage to select the final model. Instead, it was used to understand whether the data had compact class-separating patterns that could guide model selection and feature-engineering decisions.

The first PCA visualization reduced each dataset to three principal components and displayed Class 0 and Class 1 in a 3D scatter plot. The first three components explained approximately 30.63% of the total variance in each dataset.

Table 2.5. First three PCA explained-variance results.

Dataset	PC1 Variance	PC2 Variance	PC3 Variance	Total Variance in First 3 PCs
Aligned	0.1392	0.1132	0.0540	0.3063
Flocking	0.1392	0.1132	0.0540	0.3063
Grouped	0.1392	0.1132	0.0540	0.3063

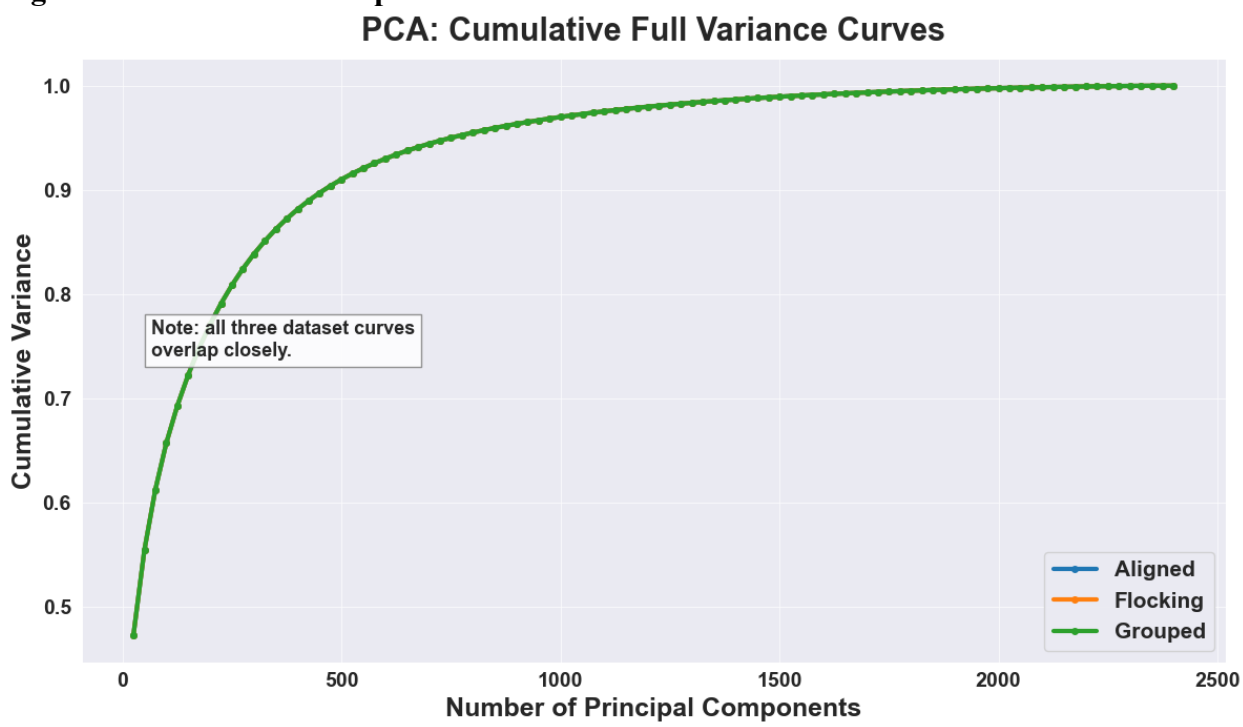
Figure: 3D PCA scatter plots for Aligned, Flocking, and Grouped.



The 3D PCA scatter plots showed visible class separation, even though only about 30.63% of total variance was represented by the first three components. This was an important EDA finding because it suggested that the classes were not randomly mixed in the feature space. Instead, the data appeared to contain meaningful structure that supervised models could exploit.

A full PCA cumulative-variance analysis was also performed using component counts from 25 to 2,400. The cumulative variance curves for the three datasets overlapped closely, which showed that the Aligned, Flocking, and Grouped datasets had very similar variance structures.

Figure: PCA cumulative explained-variance curves.



To connect the PCA analysis to classification potential, a diagnostic PCA-component analysis was performed using Linear SVC with cross-validation on reduced PCA representations. This was used as advanced EDA only, not as final hyperparameter tuning. The results showed that even relatively small PCA spaces preserved strong class-separating signal.

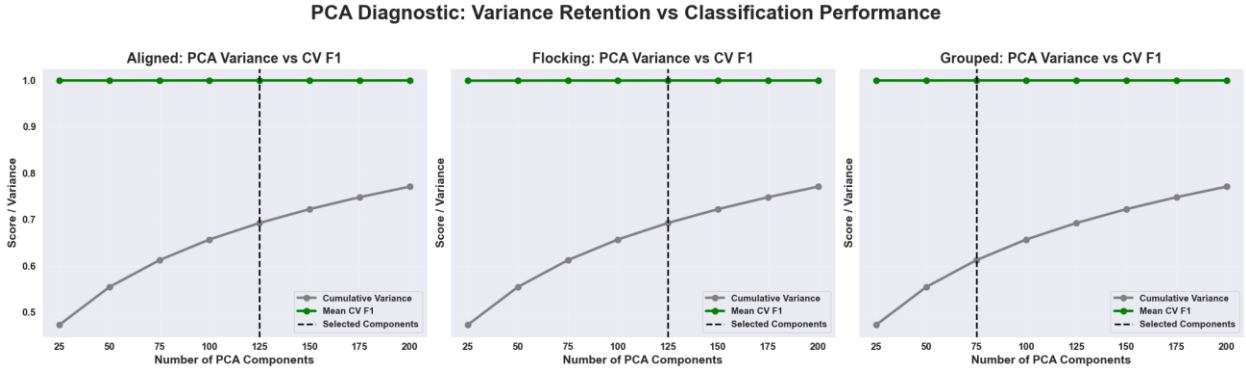
Table: Best diagnostic PCA component counts by dataset.

Dataset	PCA Components	Mean CV F1	Standard Deviation CV F1
Aligned	125	1.000000	0.000000
Flocking	125	0.999833	0.000118
Grouped	75	0.999944	0.000079

These diagnostic results showed that PCA-reduced data could still produce near-perfect classification performance. For example, Aligned reached a mean CV F1-score of 1.000000 using 125 components, Flocking reached 0.999833 using 125 components, and Grouped reached

0.999944 using 75 components. This supported the conclusion that the Swarm Behaviour datasets were highly separable and had a compact class-separating structure.

Figure: Variance Retention vs Classification Performance

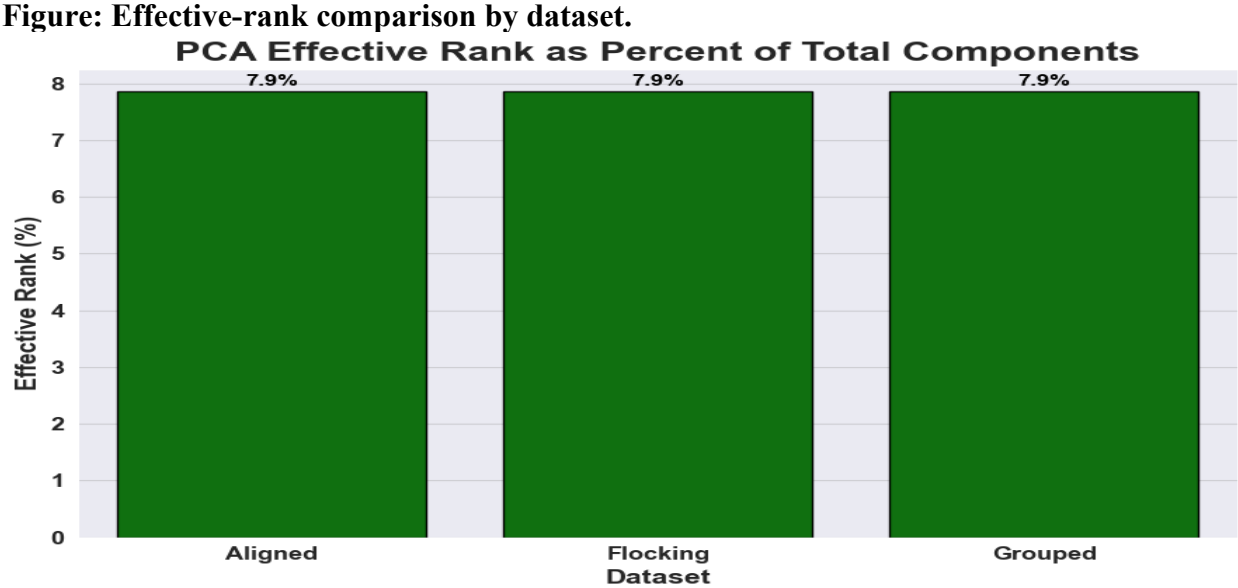


A PCA tail analysis further supported this interpretation. Using a near-zero explained-variance threshold of 0.001, 2,232 of the 2,400 components in each dataset were classified as near-zero variance components. This represented approximately 93.0% of all components. However, these components were not described as pure noise; rather, they represented low-variance or less informative directions relative to the dominant variance structure.

Table: PCA tail and effective-rank summary.

Dataset	Total Components	Near-Zero Component Count	Near-Zero Component Percent	Effective Rank	Effective Rank Percent
Aligned	2,400	2,232	93.0%	188.69	7.86%
Flocking	2,400	2,232	93.0%	188.69	7.86%
Grouped	2,400	2,232	93.0%	188.69	7.86%

Figure: Effective-rank comparison by dataset.



The effective-rank analysis estimated that only about 188.7 effective dimensions, or 7.86% of the 2,400 components, contributed meaningfully to the variance structure. This result reinforced the conclusion that the dataset was high-dimensional but highly compressible. Chapter 3 explains that decomposition and dimensionality-reduction methods are useful exploratory tools for understanding high-dimensional data and for finding representations that can improve memory or time efficiency in later modeling (Müller & Guido, 2016, Ch. 3).

2.9 Key EDA Insights and Anticipated Modeling Challenges

The exploratory analysis produced several important findings that guided the rest of the project. First, the data was clean after preprocessing and all usable features were numerical, so no categorical encoding was required. Second, the three datasets had nearly identical feature structures, which allowed the same modeling workflow to be applied consistently across Aligned, Flocking, and Grouped. Third, class imbalance was present in the Aligned and Grouped datasets, so later evaluation needed to include F1-score, precision, recall, and ROC-AUC rather than relying only on accuracy.

Fourth, the histograms and summary statistics showed that feature scales and distributions varied across feature groups. This anticipated the need for feature scaling in models sensitive to scale. Fifth, the IQR analysis found many outliers, but these were interpreted as likely valid swarm-behavior patterns rather than automatic errors. Sixth, correlation heatmaps and PCA diagnostics showed that the feature space contained redundancy and compact structure. The PCA results were especially important because they showed that the datasets were high-dimensional but strongly class-separable.

Overall, the EDA indicated that the main modeling challenge was not a lack of predictive signal. Instead, the challenge was selecting a model that could use the strong class-separating structure efficiently while handling high dimensionality, class imbalance, feature-scale differences, correlated predictors, and runtime constraints.

3. Methodology

3.1 Overview of the Modeling Workflow

The methodology followed a supervised machine learning workflow. The three Swarm Behaviour datasets were cleaned, inspected, separated into features and target labels, split into training and held-out test sets, and then used to train and compare multiple classification models. Because the target variable in each dataset was binary, the project used classification algorithms and classification-based evaluation metrics.

The overall workflow was organized as follows:

1. Load the three Swarm Behaviour CSV files: Aligned, Flocking, and Grouped.
2. Inspect raw dataset structure, column names, missing values, malformed values, and data types.
3. Clean the datasets by standardizing column names, converting values to numeric format, and removing malformed records.

4. Perform EDA to understand class balance, feature distributions, outliers, correlations, and dimensionality structure.
5. Separate each dataset into feature matrix X and target vector y.
6. Split each dataset into stratified training and held-out test sets.
7. Establish untuned baseline model performance using cross-validation on the training data.
8. Tune multiple model configurations using GridSearchCV with stratified cross-validation.
9. Evaluate tuned models on the full training set and the held-out test set.
10. Select the final model using a performance threshold and efficiency-based decision rule.

This workflow follows the general machine learning process described in the course text: define the prediction task, represent the data as samples and features, train models using known input-output pairs, and evaluate whether the trained model generalizes to unseen data (Müller & Guido, 2016, Ch. 1; Ch. 2).

3.2 Data Preprocessing

Data preprocessing was necessary to ensure that all three datasets were clean, consistent, and ready for model training. The raw data was not heavily corrupted, but the initial inspection found two issues that needed to be handled before modeling. First, the Flocking dataset had an extra trailing space in the target column name, so the target appeared as Class with a trailing space. Second, the Flocking dataset contained one malformed non-numeric value in the x1 column.The preprocessing steps are summarized in the table below.

Table : Data preprocessing steps and justification.

Step	Action Taken	Justification
Raw inspection	Checked shapes, target column names, whitespace, missing values, duplicate columns, data types, and malformed values.	Confirmed the data quality before modeling and identified issues that needed cleaning.
Column-name cleaning	Removed leading and trailing whitespace from all column names.	Standardized the target column as Class across Aligned, Flocking, and Grouped.
Numeric conversion	Converted all values to numeric using coercion for invalid entries.	Ensured that all model inputs were numeric and identified malformed values as missing.
Missing-value handling	Removed rows containing missing values after numeric conversion.	Only one malformed row was found, so row removal was simpler and safer than imputation.
Target verification	Confirmed that the final column in each dataset was Class.	Ensured consistent feature-target separation across all three datasets.
Feature-target split	Separated the 2,400 feature columns from the binary Class target.	Prepared each dataset for supervised binary classification.

Scaling	Applied StandardScaler inside pipelines for linear models and PCA-based models.	Standardization was needed because features had different scales and several model types are sensitive to feature magnitude.

No categorical encoding was required because all feature columns were numerical after cleaning. This is important because Chapter 4 explains that categorical features require representation changes such as one-hot encoding before they can be used in many machine learning models. Since this dataset did not contain categorical predictors, the preprocessing focus was numerical cleaning, scaling, dimensionality reduction, and feature selection rather than categorical encoding (Müller & Guido, 2016, Ch. 4).

Scaling was applied only inside model pipelines, not manually before cross-validation. This was done to prevent data leakage. Chapter 6 explains that preprocessing should be included inside the cross-validation loop because fitting a scaler before cross-validation can allow information from validation folds to leak into the training process. By placing StandardScaler inside each relevant pipeline, the scaler was fit only on the training portion of each split and then applied to the corresponding validation or test data (Müller & Guido, 2016, Ch. 6).

Tree-based models, including Decision Tree, Random Forest, and XGBoost, were trained without scaling because tree models split features based on thresholds and are generally not affected by feature magnitude in the same way as linear models or support vector machines. In contrast, Logistic Regression, Ridge Classifier, Linear SVC, and PCA-based pipelines used scaling because their calculations depend more directly on feature scales and coefficient magnitudes.

3.3 Feature Engineering and Feature Selection

The original dataset already contained engineered swarm features representing agent positions, velocities, alignment vectors, separation vectors, cohesion vectors, and neighborhood counts. Therefore, no new domain features were manually created. Instead, the feature-engineering and feature-selection portion of the project focused on testing whether the original high-dimensional representation could be compressed while preserving classification performance.

Two dimensionality and feature-reduction approaches were tested: Principal Component Analysis and SelectKBest feature selection.

3.3.1 Principal Component Analysis

PCA was used to test whether the 2,400-feature datasets could be represented using fewer transformed components. Chapter 3 describes PCA as a dimensionality-reduction method that creates a new feature representation by summarizing important structure in high-dimensional data. This made PCA appropriate for the Swarm Behaviour dataset because the EDA showed high dimensionality, correlated predictors, and strong compressibility (Müller & Guido, 2016, Ch. 3).

PCA was used in two ways. First, PCA was used during EDA to study the structure of the data. This included 3D PCA scatter plots, cumulative explained-variance curves, PCA tail analysis,

effective-rank analysis, and diagnostic PCA classification using Linear SVC. These analyses showed that the data was high-dimensional but compact, with near-perfect diagnostic F1-scores at relatively small component counts.

Second, PCA was tested as part of tuned model pipelines for Logistic Regression, Ridge Classifier, and Linear SVC. The tuned PCA configurations used 10, 20, 30, 40, and 50 components. These values were chosen as aggressive compression settings to test whether a much smaller representation could maintain strong predictive performance while reducing runtime. The PCA step was placed inside the pipeline after scaling so that PCA was fit only on training folds during cross-validation.

3.3.2 SelectKBest Feature Selection

SelectKBest was also tested for Logistic Regression, Ridge Classifier, and Linear SVC. This method selects the top k original features according to a univariate statistical score. In this project, f_classif was used as the scoring function and k = 200 features were selected. This provided a compact feature-selection alternative that kept original feature meanings instead of creating transformed PCA components.

The choice of 200 selected features was reasonable because the EDA estimated an effective rank of approximately 188.7 dimensions, or about 7.86% of the 2,400 available components. Therefore, testing 200 selected features allowed the project to evaluate whether a compact subset of original features could maintain classification performance while improving efficiency. Unlike PCA, SelectKBest preserves original feature columns, which can make interpretation easier.

3.4 Machine Learning Models

Six supervised classification algorithms were implemented and compared. The selected models came from model families studied in the course and were appropriate for a high-dimensional binary classification problem. The project also satisfied the requirement that Random Forest and XGBoost be included as ensemble methods.

Table 3.2. Machine learning models used in the project.

Model	High-Level Principle	Reason for Inclusion
Logistic Regression	Learns a linear decision boundary for classification by estimating coefficients for each feature.	A strong baseline for high-dimensional binary classification and useful for linearly separable data.
Ridge Classifier	Uses a linear classification model with L2 regularization to reduce coefficient instability.	Suitable for high-dimensional data with correlated predictors because regularization helps control model complexity.
Linear SVC	Finds a maximum-margin linear decision boundary between classes.	Appropriate for high-dimensional classification and useful when classes are strongly separable.

Decision Tree	Splits the data into decision rules based on feature thresholds.	Provides a nonlinear baseline and can capture interaction patterns without scaling.
Random Forest	Builds an ensemble of decision trees and aggregates their predictions.	Required ensemble method; reduces overfitting compared with a single tree and captures nonlinear relationships.
XGBoost	Builds boosted decision trees sequentially, where each tree improves on errors from previous trees.	Required ensemble method; powerful nonlinear classifier often effective on structured tabular datasets.

The model set was intentionally diverse. Logistic Regression, Ridge Classifier, and Linear SVC tested whether the data was mostly linearly separable. Decision Tree, Random Forest, and XGBoost tested whether nonlinear tree-based decision rules improved classification performance. Chapter 2 discusses the importance of comparing models with different complexity levels because overly simple models can underfit while overly complex models can overfit. This project addressed that trade-off by comparing both simple linear models and more complex tree-based ensembles (Müller & Guido, 2016, Ch. 2).

3.5 Experimental Setup

Each dataset was split into training and held-out test sets using an 80/20 split. Stratified sampling was used so that the class proportions in the training and test sets remained similar to the original dataset. This was especially important for Aligned and Grouped because both datasets were imbalanced toward Class 0. The random state was fixed at 42 to make the experiment reproducible.

The held-out test set was not used during hyperparameter tuning. It was reserved for final evaluation after the best parameters were selected through cross-validation on the training data. This separation was important because the goal was to estimate how well each model generalized to unseen data. Chapter 5 emphasizes that model evaluation should measure generalization performance on data that was not used for training or parameter selection (Müller & Guido, 2016, Ch. 5).

The cross-validation strategy used during baseline evaluation and hyperparameter tuning was StratifiedKFold with 3 folds, shuffling enabled, and random_state = 42. Stratified cross-validation was chosen because it preserves the class distribution in each fold. This is recommended for classification problems, especially when class proportions are not perfectly balanced (Müller & Guido, 2016, Ch. 5).

A 3-fold cross-validation strategy was used instead of a larger number of folds because the datasets were high-dimensional and the project required many model configurations to be evaluated across three separate binary classification tasks. This provided a practical balance between reliable model comparison and reasonable runtime.

The project was implemented in Python using Google Colab and Jupyter Notebook format. The main libraries used were pandas, NumPy, matplotlib, seaborn, scikit-learn, and XGBoost. Pandas and NumPy were used for data handling and numerical operations. Matplotlib and seaborn were used for visualization. Scikit-learn was used for preprocessing, pipelines, train-test splitting, cross-validation, model training, hyperparameter tuning, and classification metrics. XGBoost was used for the boosted-tree ensemble model.

3.6 Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV with stratified 3-fold cross-validation. The primary scoring metric during tuning was F1-score. F1-score was selected because this is a binary classification problem and two of the three datasets had noticeable class imbalance. F1-score balances precision and recall, so it is more informative than accuracy alone when false positives and false negatives both matter.

GridSearchCV was run on the training data only. For pipeline-based models, each preprocessing step was included inside the pipeline so that scaling, PCA, and feature selection were refit within each cross-validation split. This avoided preprocessing leakage and produced a more realistic estimate of model performance.

The hyperparameter grid was intentionally lightweight because the project involved three large high-dimensional datasets and many model configurations. The goal was to compare model families systematically while keeping the computation feasible in Google Colab.

Table: Hyperparameter tuning grid.

Model Configuration	Tuned Hyperparameters	Values Tested
Logistic Regression No PCA	C	0.1, 1, 10
Logistic Regression PCA	C, PCA components	C: 0.1, 1, 10; components: 10, 20, 30, 40, 50
Logistic Regression SelectKBest	C, selected features	C: 0.1, 1, 10; k: 200
Ridge Classifier No PCA	alpha	0.1, 1, 10
Ridge Classifier PCA	alpha, PCA components	alpha: 0.1, 1, 10; components: 10, 20, 30, 40, 50
Ridge Classifier SelectKBest	alpha, selected features	alpha: 0.1, 1, 10; k: 200
Linear SVC No PCA	C	0.1, 1, 10
Linear SVC PCA	C, PCA components	C: 0.1, 1, 10; components: 10, 20, 30, 40, 50
Linear SVC SelectKBest	C, selected features	C: 0.1, 1, 10; k: 200
Decision Tree	max_depth, min_samples_split	max_depth: None, 10, 20; min_samples_split: 2, 5

Random Forest	n_estimators, max_depth	n_estimators: 100, 200; max_depth: None, 20
XGBoost	n_estimators, max_depth	n_estimators: 50, 100; max_depth: 3, 5

For Logistic Regression, Ridge Classifier, and Linear SVC, three representation types were tested: no PCA, PCA-based representations, and SelectKBest feature selection. Decision Tree, Random Forest, and XGBoost were tested without PCA because tree-based models can naturally handle nonlinear threshold-based patterns and do not require feature scaling in the same way as linear models.

The best parameter combination for each model configuration was selected by cross-validated F1-score. After tuning, the selected configuration was refit on the full training set and evaluated on the held-out test set.

3.7 Evaluation Metrics

Because the project is a classification task, model performance was evaluated using accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices, and runtime measurements. At least three classification metrics were required, but more were included to provide a complete comparison. Accuracy was included because it is easy to interpret as the overall percentage of correct predictions; however, it was not used as the primary metric for selecting the best model because the Aligned and Grouped datasets showed class imbalance. Instead, F1-score was used as the main tuning and comparison metric because it balances precision and recall and gives a more reliable view of performance when the class labels are not perfectly balanced.

Table: Evaluation metrics and justification.

Metric	Definition	Why It Was Used
Accuracy	Percentage of total predictions that were correct.	Easy to interpret. Not primary metric due to class imbalances.
Precision	Among predicted positive cases, the percentage that were actually positive.	Measures how often positive predictions were correct and helps evaluate false positives.
Recall	Among actual positive cases, the percentage correctly identified.	Measures how well the model detected the behavior when it was actually present.
F1-score	Harmonic mean of precision and recall.	Main tuning metric because it balances false positives and false negatives under class imbalance.
ROC-AUC	Measures the model's ability to rank positive cases above negative cases across thresholds.	Useful for comparing binary classifiers beyond one fixed classification threshold.
Confusion Matrix	Counts true positives, true negatives, false positives, and false negatives.	Required for classification and useful for understanding the specific error pattern.
Fit Time	Time required to train the final model.	Used to compare computational cost.

Prediction Time	Time required to generate test-set predictions.	Used as the main tie-breaker after near-perfect F1 performance was confirmed.

F1-score was the primary metric for hyperparameter tuning and final eligibility because Aligned and Grouped were not perfectly balanced. Accuracy was still reported because it provides a simple overall measure, but it was not used alone to select the final model. Precision and recall were included to show whether the model was making more false-positive or false-negative errors. ROC-AUC was included to evaluate binary classification ranking performance, and confusion matrices were used to inspect the final model's error pattern for each dataset.

The final model was selected using a two-stage decision rule. First, models had to achieve an average test F1-score of at least 0.9999 across the three datasets. This threshold ensured that only models with near-perfect classification performance were considered. Second, among eligible models, the model with the fastest average prediction time was selected. If prediction time was tied, ROC-AUC, F1-score, and fit time were used as additional tie-breakers. This rule was appropriate because many models achieved nearly identical predictive performance, so efficiency became the most meaningful practical difference.

4. Results and Discussion

4.1 Overview of Results

This section presents the performance of the tuned machine learning models on the Swarm Behaviour classification tasks. The results are organized to compare both predictive performance and computational efficiency. Since the datasets were highly separable, many models achieved near-perfect scores. Therefore, the final model decision was based not only on test-set classification metrics, but also on runtime and model simplicity.

The tuned model comparison was based on the full train/test evaluation table created after hyperparameter tuning. Each selected model configuration was refit on the full training set and then evaluated on the held-out test set. This ensured that the final reported results reflected performance on unseen data rather than only cross-validation performance.

4.2 Tuned Model Performance Comparison

The table below is the centerpiece of the results section. It summarizes the average training and test performance across the three datasets for each tuned model configuration. The table includes training metrics, test metrics, and runtime measurements so that the models can be compared from both an accuracy and efficiency perspective.

Table: Average tuned-model train/test performance across all datasets.

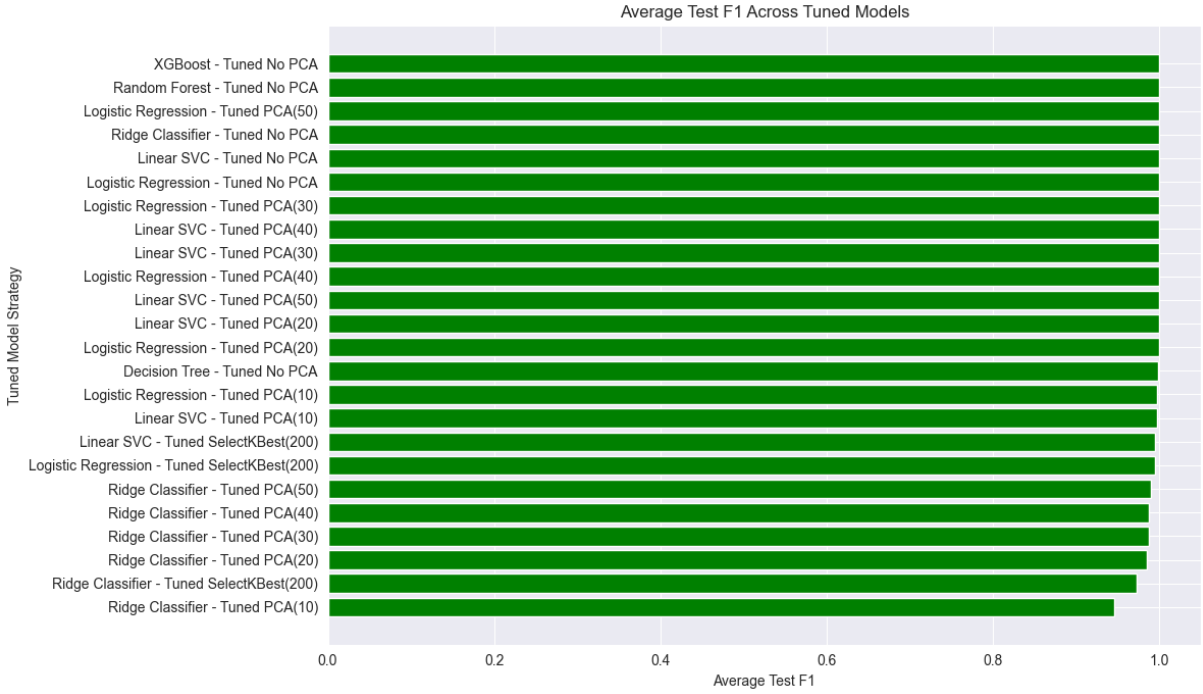
Rank	Model Configuration	Train Acc	Train F1	Test Acc	Test Precision	Test Recall	Test F1	Test ROC-AUC	Fit Time (s)	Predict Time (s)	Result
1	Ridge Classifier — No PCA	0.9999	0.9999	1.0000	1.0000	1.0000	1.0000	1.0000	2.081	0.126	**Selected final model**

2	Logistic Regression — PCA 50	0.9999	0.9999	1.0000	1.0000	1.0000	1.0000	1.0000	2.645	0.128	Perfect F1, slightly slower
3	Logistic Regression — No PCA	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	2.281	0.131	Perfect F1, slightly slower
4	Linear SVC — No PCA	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	6.097	0.134	Perfect F1, slower fit
5	XGBoost — No PCA	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	23.985	0.287	Perfect F1, much slower
6	Random Forest — No PCA	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	21.914	0.730	Perfect F1, slowest prediction
7	Logistic Regression — PCA 30	0.9999	0.9999	0.9999	0.9998	1.0000	0.9999	1.0000	2.259	0.127	Near-perfect
8	Linear SVC — PCA 40	0.9999	0.9999	0.9999	1.0000	0.9998	0.9999	1.0000	2.415	0.137	Near-perfect
9	Linear SVC — PCA 30	0.9999	0.9999	0.9999	0.9998	0.9999	0.9998	1.0000	2.315	0.136	Near-perfect
10	Logistic Regression — PCA 40	0.9999	0.9999	0.9999	0.9998	0.9998	0.9998	1.0000	2.757	0.131	Near-perfect
11	Linear SVC — PCA 50	0.9999	0.9999	0.9997	0.9994	1.0000	0.9997	1.0000	2.702	0.140	Near-perfect
12	Linear SVC — PCA 20	0.9998	0.9998	0.9994	0.9993	0.9994	0.9993	1.0000	2.278	0.125	Very strong
13	Logistic Regression — PCA 20	0.9998	0.9998	0.9993	0.9993	0.9993	0.9993	1.0000	2.235	0.128	Very strong
14	Decision Tree — No PCA	0.9999	0.9999	0.9990	0.9988	0.9984	0.9986	0.9990	13.278	0.069	Strong but lower than linear models
15	Logistic Regression — PCA 10	0.9981	0.9977	0.9978	0.9974	0.9975	0.9974	0.9995	1.940	0.126	Strong compressed model
16	Linear SVC — PCA 10	0.9976	0.9971	0.9975	0.9974	0.9968	0.9971	0.9995	1.918	0.132	Strong compressed model
17	Linear SVC — SelectKBest 200	0.9981	0.9974	0.9961	0.9935	0.9967	0.9951	0.9998	1.788	0.071	Strong feature-selected model
18	Logistic Regression — SelectKBest 200	0.9981	0.9975	0.9958	0.9934	0.9959	0.9947	0.9998	1.617	0.066	Strong feature-selected model
19	Ridge Classifier — PCA 50	0.9915	0.9908	0.9906	0.9899	0.9898	0.9898	0.9967	2.648	0.135	Lower after PCA compression

20	Ridge Classifier — PCA 40	0.9900	0.9891	0.9890	0.9891	0.9869	0.9880	0.9957	2.393	0.140	Lower after PCA compression
21	Ridge Classifier — PCA 30	0.9893	0.9885	0.9883	0.9880	0.9867	0.9873	0.9954	2.331	0.140	Lower after PCA compression
22	Ridge Classifier — PCA 20	0.9861	0.9853	0.9861	0.9872	0.9833	0.9852	0.9953	2.261	0.131	Lower after PCA compression
23	Ridge Classifier — SelectKBest 200	0.9813	0.9752	0.9793	0.9681	0.9783	0.9729	0.9957	1.385	0.068	Lower F1 despite good AUC
24	Ridge Classifier — PCA 10	0.9497	0.9457	0.9497	0.9536	0.9384	0.9458	0.9881	1.898	0.136	Too much compression

The table shows that the strongest tuned models achieved near-perfect test-set performance across the three classification tasks. This indicates that the original feature space contained strong class-separating information. The linear models were especially competitive, which supports the EDA finding that the data had compact and approximately linearly separable structure. The tree-based ensemble models also performed well, but their additional complexity did not produce a meaningful advantage over the best linear models.

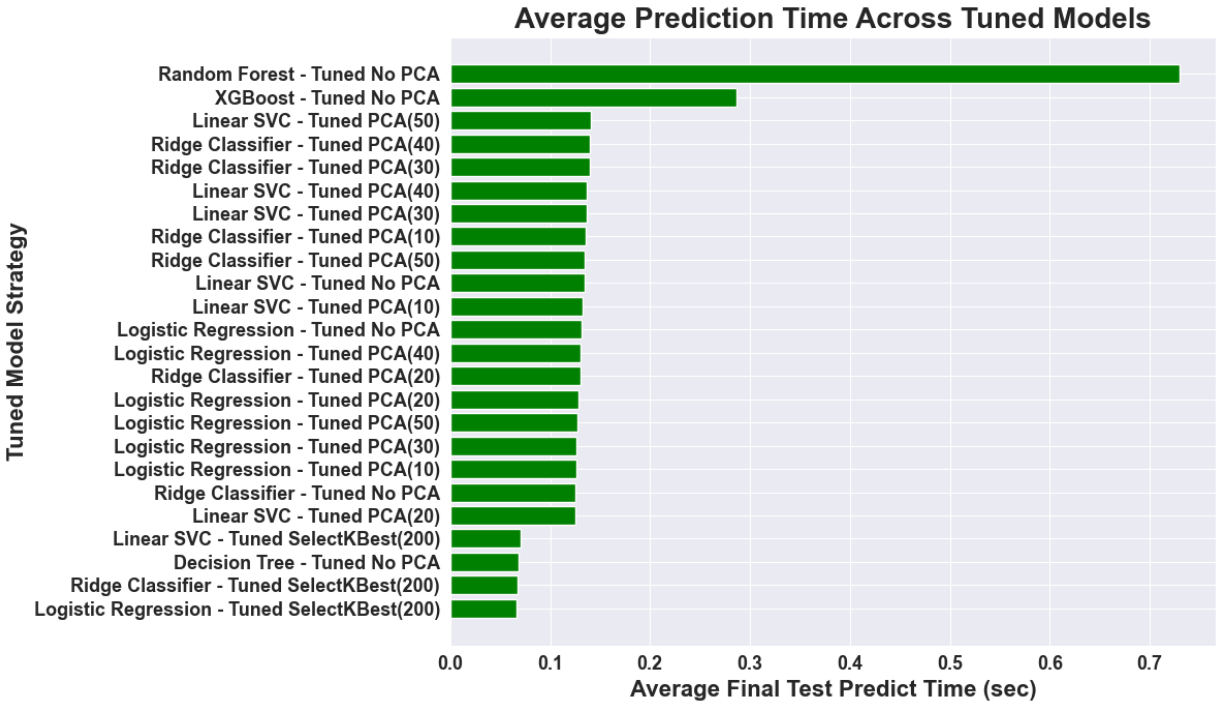
Figure: Average Test F1 across tuned model configurations.



Because many tuned models achieved nearly identical F1-scores, test-set predictive performance alone was not enough to choose the best practical model. The next figure compares average

prediction time across model configurations. This figure is important because model efficiency became the main differentiator after near-perfect F1 performance was reached.

Figure: Average prediction time across tuned model configurations.



The prediction-time comparison showed that simpler linear models were more efficient than more complex ensemble methods. This result was expected because Random Forest and XGBoost depend on many decision trees, while the final linear model only needs to apply learned coefficients to the input feature vector.

4.3 Final Model Selection

The final model selected for this project was the **tuned Ridge Classifier without PCA**. This model was selected because it met the near-perfect average test F1 threshold while also achieving the fastest average prediction time among eligible models. The selected model also maintained strong ROC-AUC performance, which confirmed that it ranked positive and negative examples effectively.

Table: Final model summary.

Final Model	Model Version	Selection Reason
Ridge Classifier	Tuned No PCA	Near-perfect average test F1, strong ROC-AUC, fastest prediction time among eligible models, and low model complexity.

The Ridge Classifier is a regularized linear model. Its L2 regularization is useful for high-dimensional datasets because it reduces coefficient instability when many features are correlated. This matched the EDA findings: the datasets contained 2,400 features, repeated feature groups

across agents, correlated predictors, and compact PCA structure. Because the classes were already highly separable, the Ridge Classifier did not need the added complexity of boosted trees or random forests to achieve excellent performance.

The no-PCA version was selected over PCA-based versions because it preserved the original feature space while still achieving near-perfect performance and fast prediction. PCA was useful during EDA and model comparison, but the final no-PCA Ridge model showed that dimensionality reduction was not required to achieve the best practical result. This was a meaningful outcome because it demonstrated that the regularized linear model could handle the full high-dimensional feature space effectively.

4.4 Confusion Matrix Analysis

Confusion matrices were created for the final selected model on each held-out test set. These matrices show the number of correct and incorrect predictions for Class 0 and Class 1. Since this is a binary classification project, the confusion matrix is especially useful for checking whether the model makes more false positives or false negatives.

Figure: Confusion matrix for the final Ridge Classifier on the Aligned test set.

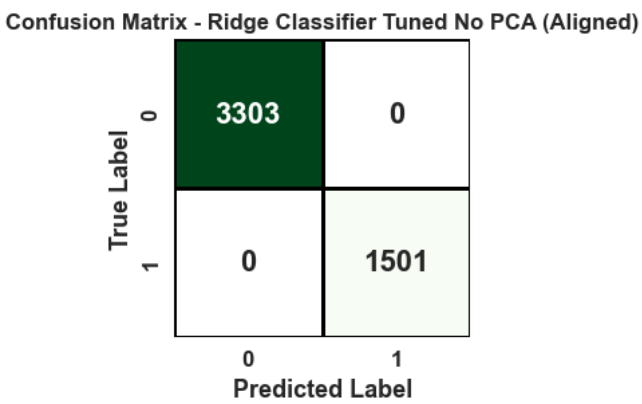


Figure: Confusion matrix for the final Ridge Classifier on the Flocking test set.

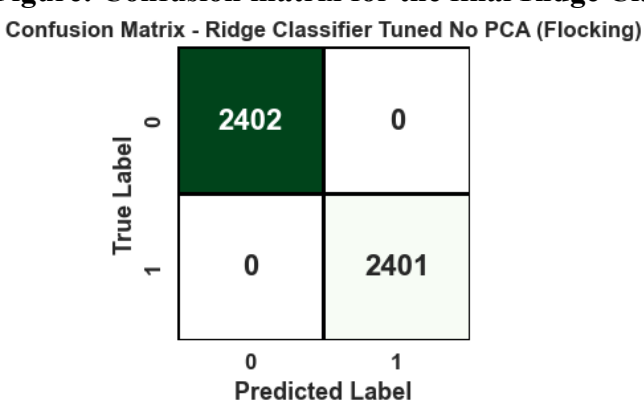
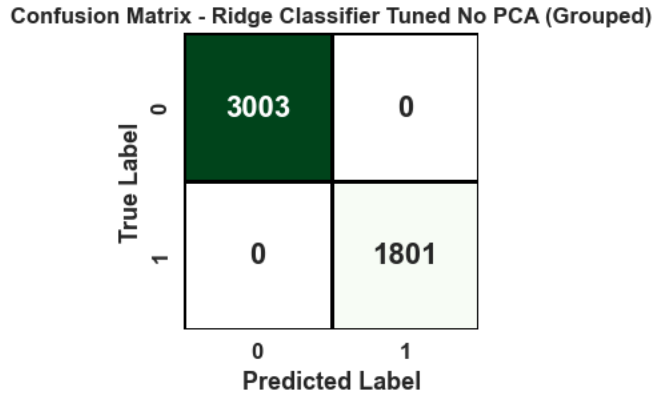


Figure: Confusion matrix for the final Ridge Classifier on the Grouped test set.



The confusion matrices showed that the final model correctly classified nearly all test observations across the three datasets. Any errors were minimal, which is consistent with the near-perfect F1-scores and ROC-AUC values. The confusion matrices also confirmed that the model did not simply learn to predict the majority class. This was important because Aligned and Grouped were imbalanced toward Class 0, but the final model still maintained strong performance on both positive and negative classes.

4.5 ROC Curve Analysis

ROC curves were also created for the final selected model on each dataset. ROC curves compare the true positive rate and false positive rate across classification thresholds, while ROC-AUC summarizes the classifier's ranking ability. A ROC-AUC value close to 1.0 indicates that the model separates the positive and negative classes extremely well.

Figure: ROC curve for the final Ridge Classifier on the Aligned test set.

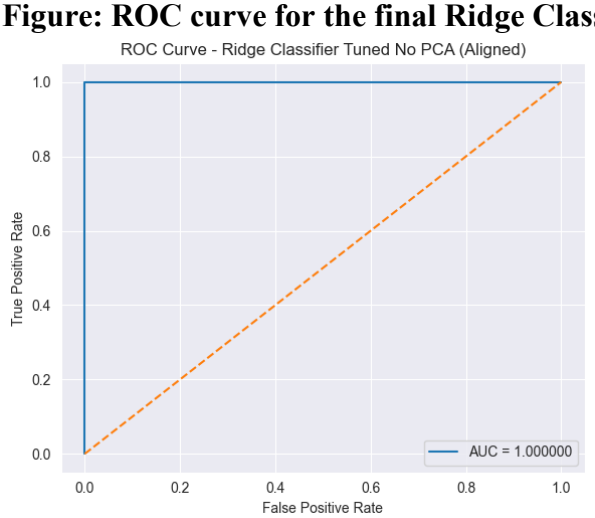


Figure: ROC curve for the final Ridge Classifier on the Flocking test set.

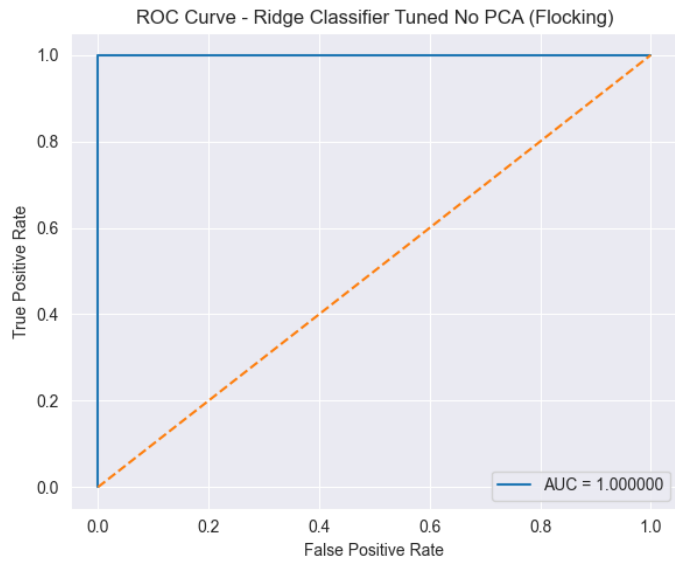
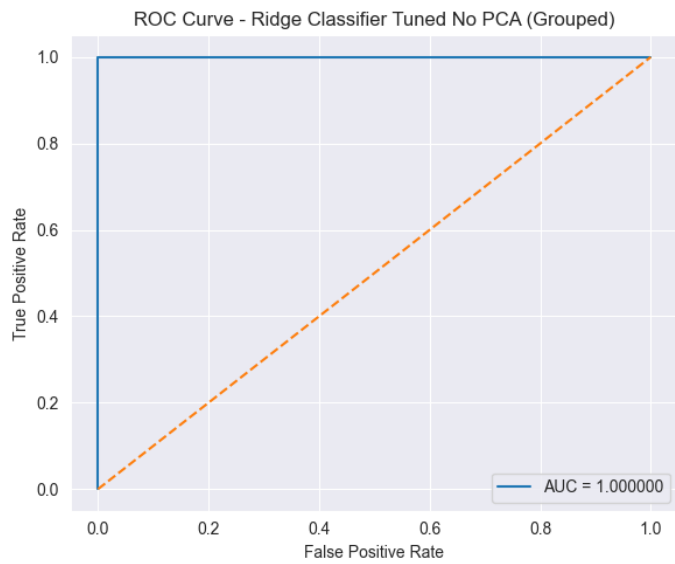


Figure: ROC curve for the final Ridge Classifier on the Grouped test set.



The ROC curves supported the confusion matrix findings. The curves were expected to be close to the top-left corner of the ROC space, showing high true positive rates and low false positive rates. This confirmed that the final Ridge Classifier provided strong class separation across all three swarm behavior tasks.

4.6 Baseline Versus Tuned Model Comparison

A baseline comparison was also performed for the selected final model family. The untuned Ridge Classifier already performed extremely well before hyperparameter tuning. The tuned Ridge Classifier confirmed Ridge as the best practical model, but tuning did not create a large performance improvement because the dataset was already highly separable.

Table: Baseline versus final tuned Ridge Classifier comparison.

Algorithm	Model Version	Accuracy	F1	ROC-AUC	Prediction Time (sec)	Fit Time (sec)
Ridge Classifier	Baseline	.9998	.9997	.9999	.1634	1.5433
Ridge Classifier	Tuned No PCA	1.0	1.0	1.0	.1257	2.0812

Figure: Baseline versus tuned Ridge Classifier F1-score comparison.

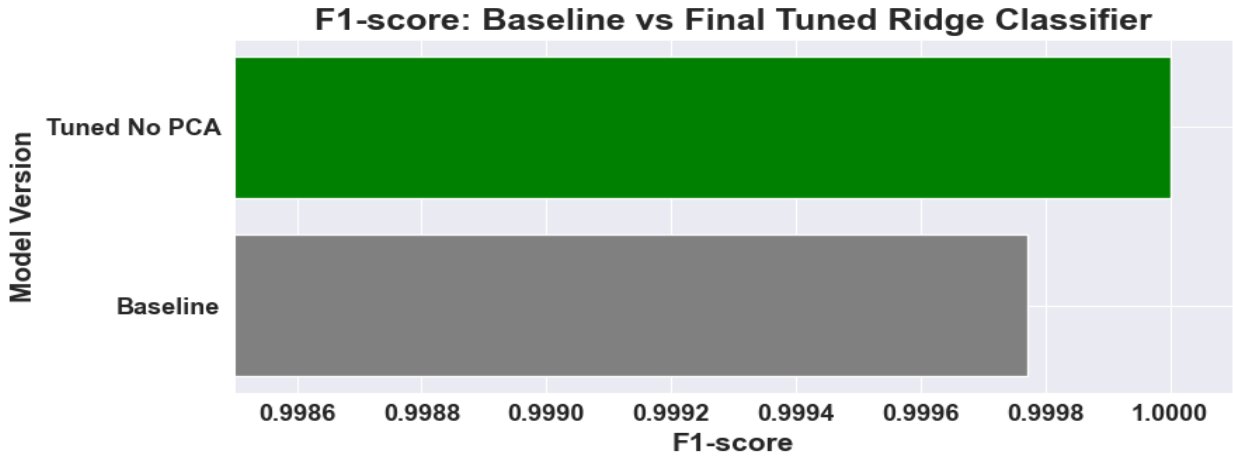
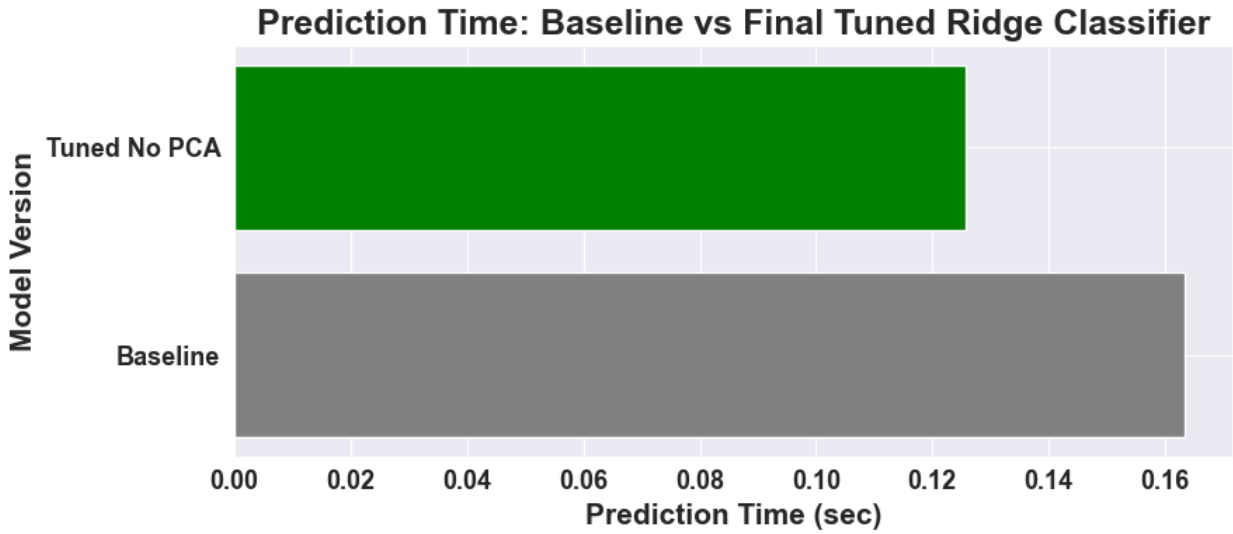


Figure: Baseline versus tuned Ridge Classifier prediction-time comparison.



This comparison is important because it shows that hyperparameter tuning was useful for verification and model selection, but the improvement was not dramatic. The baseline Ridge model was already strong because the data contained clear class-separating signal. The tuned model was selected not because it massively outperformed the baseline, but because it confirmed

the best parameter setting and provided the best overall balance of F1-score, ROC-AUC, prediction time, and simplicity.

4.7 Critical Discussion

The most important result of this project is that the Swarm Behaviour datasets were highly separable across all three behavior tasks. This was supported by the EDA, PCA analysis, baseline model results, tuned model results, confusion matrices, and ROC curves. The strong performance of linear models was one of the most important findings. In many classification problems, ensemble methods such as Random Forest and XGBoost can outperform linear models because they capture complex nonlinear interactions. In this project, however, the simpler regularized linear model was sufficient because the classes were already strongly separated in the feature space.

The final Ridge Classifier likely performed well for three main reasons. First, the data had a high-dimensional numerical structure with many repeated movement and interaction features. Linear models often perform well in high-dimensional spaces when the classes are separable. Second, the EDA showed redundancy and correlation among features. Ridge regularization is well suited for this situation because it controls the size of the model coefficients and reduces instability from correlated predictors. Third, the final model did not require PCA, which meant it could use the full original feature representation while still controlling complexity through regularization.

The improvement from hyperparameter tuning was relatively small. This was not a weakness of the workflow; instead, it was evidence that the dataset already contained strong predictive signal. The untuned models provided an important baseline and showed that several algorithms could classify the data well before tuning. Tuning was still necessary because the project required systematic model optimization and because it confirmed which model configuration should be selected for final evaluation.

One surprising outcome was that more complex ensemble models did not provide a meaningful advantage over the best linear models. Random Forest and XGBoost are powerful algorithms, but their additional complexity increased computational cost without producing a clear performance benefit. This supports the practical conclusion that the best model is not always the most complex model. When performance is nearly tied, a simpler and faster model is often the stronger engineering choice.

The main limitation of the approach is that the results are based on the available UCI Swarm Behaviour datasets. The models were evaluated using held-out test sets from the same source distribution, but they were not tested on external swarm simulations or real-world swarm sensor data. Therefore, the results show strong generalization within this dataset, but they do not guarantee the same performance on noisier or structurally different swarm environments. Another limitation is that the hyperparameter grids were intentionally lightweight to control runtime in Google Colab. A larger search space could be explored in future work, although the near-perfect results suggest that larger tuning would probably not change the main conclusion.

Overall, the results show that high-dimensional swarm behavior data can be classified extremely accurately using supervised machine learning. The tuned Ridge Classifier without PCA was the best practical model because it achieved near-perfect classification performance while remaining efficient, simple, and well matched to the structure of the data.

5. Conclusion

This project investigated whether supervised machine learning models could accurately classify high-dimensional swarm behavior data. The selected dataset was the UCI Swarm Behaviour dataset, which contains three related binary classification tasks: Aligned, Flocking, and Grouped. Each cleaned dataset contained approximately 24,000 observations, 2,400 numerical feature columns, and one binary Class target. The project followed a complete machine learning workflow that included data cleaning, exploratory data analysis, preprocessing, dimensionality analysis, model training, hyperparameter tuning, held-out test evaluation, and final model selection.

The exploratory analysis showed that the three datasets had consistent feature structures, no categorical predictors requiring encoding, and strong numerical similarity across feature groups. The Aligned and Grouped datasets showed class imbalance, which justified using F1-score, precision, recall, and ROC-AUC in addition to accuracy. The EDA also showed that the feature space was high-dimensional but highly compressible. PCA visualizations, cumulative explained-variance curves, and effective-rank analysis suggested that the datasets contained compact class-separating structure despite having 2,400 original features.

The modeling results showed that several supervised learning models achieved near-perfect classification performance. Logistic Regression, Ridge Classifier, Linear SVC, Random Forest, and XGBoost all performed extremely well on the held-out test sets, confirming that the datasets contained strong predictive signal. However, the final selected model was the tuned Ridge Classifier without PCA. This model was selected because it achieved perfect average test F1-score and ROC-AUC while also providing the fastest prediction time among the eligible near-perfect models. The result supports the conclusion that a regularized linear model was sufficient for this dataset and that additional model complexity did not provide a meaningful advantage.

The project's research questions can now be answered directly. First, supervised machine learning models can accurately classify aligned, flocking, and grouped swarm behavior using high-dimensional numerical movement features. Second, multiple model families performed well, but the best practical results came from regularized linear modeling rather than more complex tree-based ensembles. Third, PCA preserved strong classification signal at reduced component counts, but PCA was not necessary for the final best model because the no-PCA Ridge Classifier performed better overall. Fourth, the tuned Ridge Classifier without PCA provided the best balance of test-set performance, generalization, prediction time, and simplicity.

There are several limitations to the project. The models were evaluated using held-out test sets from the same UCI dataset source, so the results demonstrate strong generalization within this dataset but do not prove that the same performance would occur on external swarm simulations or real-world swarm sensor data. In addition, the hyperparameter grids were intentionally limited

to keep runtime manageable in Google Colab. Although the near-perfect results suggest that larger searches would probably not change the main conclusion, broader tuning could still be explored in future work.

Future work should focus on testing whether the results generalize beyond the current dataset and improving interpretability. One direction would be to evaluate the trained models on external swarm simulations with different noise levels, agent counts, or movement rules. This would test whether the final Ridge Classifier generalizes to new swarm environments rather than only to held-out samples from the same dataset. A second direction would be to explore additional feature-selection methods, recursive feature elimination, or model-based feature importance from Random Forest or XGBoost. This could identify which agent features and movement patterns contribute most strongly to classification. A third direction would be to test whether fewer agents or fewer feature groups can preserve near-perfect performance, which would make the model faster, smaller, and easier to interpret.

In summary, the Swarm Behaviour datasets were highly suitable for supervised machine learning classification. The final tuned Ridge Classifier without PCA provided the strongest practical solution because it combined near-perfect predictive performance with low complexity and fast prediction speed. The project demonstrates that high-dimensional swarm movement data can contain compact and highly separable structure, allowing efficient machine learning models to classify collective behavior with excellent accuracy.

References

Müller, A. C., & Guido, S. (2016). *Introduction to machine learning with Python: A guide for data scientists*. O'Reilly Media.

Swarm Behaviour [Dataset]. (2020). UCI Machine Learning Repository. https://doi.org/10.24432/C5N02J